1.Insertion At Beginning

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first, *second, *tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 20;
    second->data = 30;
    tail->data = 40;

    head = first;
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
    first->next = second;
    second->next = tail;
    tail->next = head;
}

void insertbeginning(int data){
    struct node *newnode, *tail;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    tail = head;

    while(tail->next != head){
        tail = tail->next;
    }

    newnode->next = head;
    tail->next = newnode;

    head = newnode;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertbeginning(10);

```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertbeginning(10);

    printf("List: ");
    display();

    return 0;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

2.Insertion at End

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    tail->data = 30;

    head = first;
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

```c
    first->next = second;
    second->next = tail;
    tail->next = head;
}

void insertend(int data){
    struct node *newnode, *temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    temp = head;

    while(temp->next != head){
        temp = temp->next;
    }

    temp->next = newnode;
    newnode->next = head;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

```c
    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertend(40);

    printf("List: ");
    display();

    return 0;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

3.Insert at Position

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 40;

    head = first;
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
    first->next = second;
    second->next = third;
    third->next = head;
}

void insertpos(int data,int pos){
    struct node *newnode,*temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    temp = head;

    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }

    newnode->next = temp->next;
    temp->next = newnode;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

Share Run

```c
}

void display(){
    struct node *temp = head;

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertpos(30,3);

    printf("List: ");
    display();

    return 0;
}
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

4.Deletion at beginning

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

main.c

Share Run

```c

    first->next = second;
    second->next = third;
    third->next = head;
}

void deletebegin(){
    struct node *temp,*last;

    temp = head;
    last = head;

    while(last->next != head){
        last = last->next;
    }

    head = temp->next;
    last->next = head;

    free(temp);
}
```

Output

Clear

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

main.c

```c
    }

    temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}

int main(){

    beginning();

    printf("Before deletion:\n");
    display();

    deletebegin();

    printf("\nAfter deletion:\n");
    display();

    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
20 30

=== Code Execution Successful ===
```

5.Deletion at End

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c

    first->next = second;
    second->next = third;
    third->next = head;
}

void deleteend(){
    struct node *temp,*prev;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;

    while(temp->next != head){
        prev = temp;
        temp = temp->next;
    }

    prev->next = head;
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

main.c

```c

    temp = head;

    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}

int main(){

    create();

    printf("Before deletion:\n");
    display();

    deleteend();

    printf("\nAfter deletion:\n");
    display();

    return 0;
}
```

Output

```
Before deletion:
10 20 30
After deletion:
10 20

=== Code Execution Successful ===
```

6.Deletion at position

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void create(){
    struct node *first,*second,*third,*fourth;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    fourth = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;
    fourth->data = 40;
```

Output

```
Before deletion:
10 20 30 40
After deletion:
10 20 40

=== Code Execution Successful ===
```

main.c

```c
    first->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = head;
}

void deletepos(int pos){
    struct node *temp,*nextnode;
    int i;

    temp = head;

    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }

    nextnode = temp->next;
    temp->next = nextnode->next;

    free(nextnode);
}
```

Output

```
Before deletion:
10 20 30 40
After deletion:
10 20 40

=== Code Execution Successful ===
```

main.c

Share Run

```c

    temp = head;

    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}

int main(){

    create();

    printf("Before deletion:\n");
    display();

    deletepos(3);

    printf("\nAfter deletion:\n");
    display();

    return 0;
}
```

Output

```
Before deletion:
10 20 30 40
After deletion:
10 20 40

=== Code Execution Successful ==
```